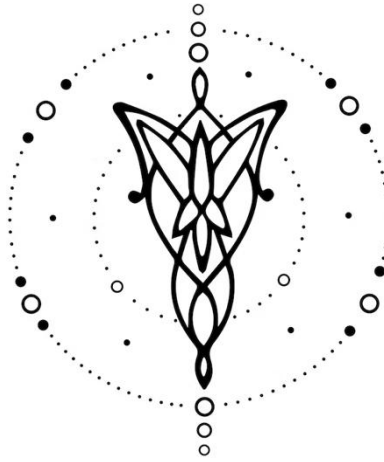




CardGameLang

Etapa III — Backend

Diseño e Implementación de un Lenguaje
Autómatas, Teoría de Lenguajes y Compiladores

NOMBRE	APELLIDOS	Legajo	E-mail
Julia	Petti Pagadizabal	62453	jpettipgadizabal@itba.edu.ar
João	Ferreira	69220	jcouceirofulgenciot@itba.edu.ar

Repositorio: <https://github.com/JuliaPettiP/CardGameLang.git>

Tabla de Contenidos

1. Introducción	3
2. Modelo Computacional	3
2.1. Dominio	3
2.2. Lenguaje	4
3. Implementación	5
3.1. Frontend	5
3.2. Backend	6
3.3. Adicionales	8
3.4. Dificultades Encontradas	8
4. Futuras Extensiones	8
5. Conclusiones	8
6. Referencias	9
7. Bibliografía	9

1. Introducción

CardGameLang es un lenguaje declarativo de dominio específico (DSL) cuyo objetivo es modelar y describir juegos de cartas por turnos. En lugar de programar un videojuego, el usuario *declara* los componentes de un juego —jugadores, mazo, cartas, reglas de jugada, reglas del turno y condición de victoria— y el compilador valida que la definición sea consistente y produce una **simulación textual** de la evolución de una partida.

Este documento describe la última etapa del proyecto, en la que se completa el *frontend* (análisis léxico y sintáctico) y se incorporan las dos fases del *backend*:

- **Análisis semántico:** a partir del AST, verifica que el programa de entrada es correcto respecto del dominio.
- **Generación de código:** produce el artefacto final de la solución. Dado que el dominio se basa en simular el comportamiento de un juego, el *runtime* es relevante y el artefacto generado es una simulación textual de una partida.

2. Modelo Computacional

2.1. Dominio

El dominio son los **juegos de cartas por turnos** en su forma simplificada. Un juego queda determinado por:

- Una **cantidad de jugadores** (un valor único o un rango).
- Un **mazo** de cartas, cada una con un nombre y atributos simples (`color`, `value`, `points`, `effect`).
- La **mano inicial** de cada jugador.
- **Reglas de validez de jugada** (`play_rule`), que indican cuándo una carta puede jugarse.
- **Reglas del juego** (`rules`), estructuras condicionales que asocian "jugar una carta" con una o más acciones a ejecutar.
- La dinámica del **turno** (`turn`) con acciones (`must`), y opcionales (`may`) o una estructura condicional `if cannot_play / else`.
- El conjunto de **acciones válidas** (`actions`).
- La **condición de victoria** (`win`): vaciar la mano (`empty_hand`) o alcanzar un puntaje objetivo (`reach_points N`).

Quedan deliberadamente fuera del alcance la inteligencia artificial, la concurrencia y las reglas extremadamente complejas; las cartas tienen atributos simples y las reglas se limitan a condiciones y acciones básicas.

2.2. Lenguaje

Un programa define **uno o más** juegos; cada juego es un bloque `game <Nombre> { ... }` cuyas secciones aparecen en un **orden fijo**:

```
players → deck → hand → play_rule → rules → turn → actions → win
```

Las secciones `play_rule`, `rules`, `turn` y `actions` pueden omitirse (son opcionales a nivel gramatical); las restantes son obligatorias. Un programa puede contener varios bloques `game` consecutivos, que se procesan de forma independiente. A continuación, un ejemplo completo inspirado en juegos tipo UNO (objetivo: vaciar la mano):

```
game ColorMatch {
  players 2..4
  deck {
    card NumberCard { color { red, blue, green, yellow } value 0..9 }
    card SkipCard    { color { red, blue, green, yellow } effect skip_next_player }
    card DrawTwoCard { color { red, blue, green, yellow } effect draw 2 }
    card WildCard    { effect change_color }
  }
  hand 7
  play_rule {
    allow if same_color
    allow if same_value
    allow if wild
  }
  rules {
    if played SkipCard    { skip_next_player }
    if played DrawTwoCard { next_player draw 2 }
    if played WildCard    { choose_color }
  }
  turn {
    if cannot_play { draw 1 }
    else { may play 1 may draw 1 }
  }
  actions { draw play choose_color }
  win if empty_hand
}
```

Y un juego por puntos (objetivo: alcanzar un puntaje):

```
game HighPoints {
  players 2
  deck {
    card LowCard { color { red, black } points 1..3 }
    card HighCard { color { red, black } points 4..7 }
    card BonusCard { color gold points 5 effect extra_turn }
  }
  hand 5
  play_rule { allow if any_card }
  rules { if played BonusCard { extra_turn } }
  turn { must play 1 }
  actions { play }
  win if reach_points 21
}
```

3. Implementación

El proyecto está escrito en C y se construye con CMake; el *frontend* se genera con Flex (analizador léxico) y Bison (analizador sintáctico). La compilación se realiza en un contenedor Docker con la toolchain GNU (`gcc` , `flex` , `bison` , `cmake` , `make`), con AddressSanitizer activado.

El flujo del compilador (`EntryPoint.c`) es:

1. Análisis sintáctico (que dispara el análisis léxico) → construye el AST.
2. Si tiene éxito, análisis semántico sobre el AST.
3. Si el programa es semánticamente válido, generación de código (resumen + simulación).
4. Liberación de recursos del AST y de cada módulo.

3.1. Frontend

Análisis léxico — `FlexPatterns.l` . Reconoce:

- Palabras clave de sección: `game` , `players` , `deck` , `card` , `hand` , `play_rule` , `rules` , `turn` , `actions` , `win` .
- Palabras clave de reglas/turno: `if` , `allow` , `played` , `cannot_play` , `else` , `may` , `must` .
- Condiciones: `empty_hand` , `reach_points` , `same_color` , `same_value` , `wild` , `any_card` .
- Atributos de carta: `color` , `value` , `points` , `effect` .
- Símbolos: `..` (rango), `{` , `}` , `,` .
- `INTEGER` (`[[[:digit:]]+`) e `IDENTIFIER` (`[a-zA-Z_][a-zA-Z0-9_]*`).
- Comentarios multilínea `/* ... */` mediante un estado exclusivo `MULTILINE_COMMENT` ; los espacios y el contenido de los comentarios se ignoran.

Análisis sintáctico — `BisonGrammar.y`. La gramática construye el AST a través de *semantic actions* (`BisonActions.c`). La raíz `Program` contiene una `GameList` (uno o más juegos); el no terminal `card game list` es recursivo y permite encadenar varios bloques `game`. El AST (`AbstractSyntaxTree.h`) modela cada `Game` y sus listas enlazadas: `CardList`, `CardAttributeList`, `ColorList`, `ActionNameList`, `PlayRuleList`, `GameRuleList`, `RuleStatementList`, `TurnActionList`, además de `PlayerRange`, `WinCondition` y `Turn`.

Algunos puntos de diseño de la gramática:

- Los argumentos enteros opcionales (`value 5` vs `value 0..9`, `effect draw 2` vs `effect skip_next_player`, `may play 1` vs `may play`) se resuelven por la preferencia de *shift* de Bison, que en estos casos es siempre la correcta.
- Una sentencia de regla como `next player draw 2` se almacena como **dos** `RuleStatement` consecutivos: `("next player", 0)` y `("draw", 2)`. El backend los reinterpreta como "el siguiente jugador roba 2".

3.2. Backend

Análisis semántico — `SemanticValidator.c`

Valida **cada juego** del programa de forma independiente y rechaza la entrada si detecta inconsistencias respecto del dominio. Verificaciones implementadas por juego:

- **Jugadores:** el rango existe y es válido (`min ≥ 1`, `max ≥ 1`, `min ≤ max`).
- **Mazo:** no vacío; nombres de carta no vacíos y **únicos**.
- **Acciones:** nombres no vacíos y **únicos** (sin duplicados).
- **Condición de victoria:** presente; si es `reach_points`, el objetivo debe ser positivo.
- **Mano inicial:** `hand > 0`.
- **Reglas de jugada** (`play_rule`): toda carta referenciada (sujeto u objetivo de `if played`) debe estar definida en el mazo.
- **Reglas del juego** (`rules`): la carta disparadora debe existir en el mazo y cada acción del cuerpo debe estar declarada en `actions`.
- **Turno:** cada acción usada (en bloque simple, `if` o `else`) debe estar declarada en `actions`.

La fase devuelve `SUCCEEDED` / `FAILED`; ante el primer error reporta un mensaje descriptivo y rechaza el programa.

Generación de código — `Generator.c`

Como el dominio es la simulación de juegos, el artefacto final tiene en cuenta el *runtime*. La fase recorre todos los juegos del programa y, para cada uno, produce dos bloques:

1. **GAME SUMMARY:** descripción estática del juego (nombre, rango de jugadores, mano inicial, cartas, acciones y condición de victoria).

2. **SIMULATION**: una **simulación textual de la evolución de una partida**, que es el objetivo central de esta etapa.

Modelo de la simulación (determinístico y reproducible):

- Se juega con `players.min` jugadores (acotado a `[1, 8]`); cada uno comienza con `hand` cartas y o puntos.
- En cada turno, el jugador activo ejecuta el bloque `turn`: en un turno simple corre sus sentencias `must` / `may`; en un turno condicional, "no puede jugar" (`cannot_play`) se modela como "mano vacía", eligiendo el bloque `if` o `else`.
- Política de acciones opcionales (`may`): el jugador **juega** siempre que puede (avanza hacia ambas condiciones de victoria) y solo **roba** si se queda sin cartas, de modo que la simulación progresa y siempre termina.
- Al jugar una carta se recorre el mazo en *round-robin*; si la carta otorga `points` se acumulan; si existe una `rule if played <carta>`, se aplican sus efectos: `next player`, `draw N`, `draw N`, `skip_next_player`, `extra turn` (el resto se narra sin alterar el estado).
- La condición de victoria se evalúa tras cada jugada: `empty hand` (un jugador llega a 0 cartas) o `reach points` (alcanza el objetivo).
- Un tope de turnos (`MAX_TURNS`) garantiza la terminación; si nadie gana, se informa que se alcanzó el límite.

Ejemplo de salida (juego `HighPoints`, abreviado):

```
=== SIMULATION ===
Players: 2, initial hand: 5 each

Turn 1 - player 1 (hand 5, points 0):
    player 1 plays LowCard (+2 points, total 2, hand 4)
...
Turn 10 - player 2 (hand 1, points 18):
    player 2 plays LowCard (+3 points, total 21, hand 0)

=== RESULT ===
Player 2 wins by reaching 21 points (turn 10).
```

Casos de prueba

Bajo `src/test/c/` hay 20 casos de **aceptación** y 19 casos de **rechazo**, ejecutados por `test.sh`. Cubren los casos propuestos en la especificación, entre ellos: juegos numéricos y por puntos, cartas con atributos y efectos, reglas basadas en la jugada anterior, turnos con `if cannot play` y con acciones opcionales, y un programa con **varios juegos**; y rechazos por programa malformado, rango de jugadores inválido, carta sin nombre, atributos/ condiciones no soportadas, acción no declarada, ausencia de condición de victoria y referencias a cartas/efectos inexistentes.

3.3. Adicionales

- **Simulación determinística:** se utiliza un generador pseudoaleatorio con semilla fija para elegir el valor de puntos dentro del rango de una carta, de modo que la salida sea reproducible y verificable.
- **Interpretación de efectos:** el backend reconoce y aplica los efectos del dominio (`next_player draw`, `skip_next_player`, `extra_turn`) en lugar de solo listarlos.

3.4. Dificultades Encontradas

- **Orden fijo de secciones:** fijar el orden `players → deck → hand → play rule → rules → turn → actions → win` simplifica la gramática y evita ambigüedades, a costa de menor flexibilidad para el usuario.
- **Conflictos shift/reduce:** los argumentos enteros opcionales generan conflictos resueltos por la preferencia de *shift* de Bison; se documentaron en la gramática para dejar explícito que la resolución por defecto es la deseada.
- **Representación de `next_player draw N`:** al no introducir un no terminal dedicado, la sentencia se parte en dos `RuleStatement`, y el backend debe reconstruir la intención ("el siguiente jugador roba N").
- **Garantizar terminación de la simulación:** con turnos condicionales (`may play` + `may draw`) una política ingenua podía no terminar; fue necesario definir una política de acciones opcionales y un tope de turnos.
- **Entorno de compilación:** el proyecto asume *toolchain* GNU; se compila en Docker porque en macOS el compilador por defecto es Clang.

4. Futuras Extensiones

- **Emparejamiento real en `play_rule`:** hoy la validez de jugada se valida estructuralmente; podría modelarse el "color/valor de la carta anterior" durante la simulación.
- **Jugadores con estrategia / IA básica y mazos barajados** con aleatoriedad configurable (semilla por parámetro).
- **Más efectos y condiciones** y un catálogo extensible de acciones.
- **Exportar la simulación** a otros formatos (JSON, traza estructurada) además de texto.

5. Conclusiones

Se completó el ciclo del compilador para CardGameLang: análisis léxico y sintáctico que construyen un AST, una fase de análisis semántico que garantiza la consistencia del juego respecto del dominio, y una fase de generación de código que produce el artefacto

relevante para este dominio: una simulación textual de una partida. El conjunto de casos de aceptación y rechazo respalda que el lenguaje cumple las construcciones propuestas en la especificación. El diseño declarativo y las validaciones permiten detectar errores en la definición de un juego antes de cualquier implementación más completa.

6. Referencias

- Código fuente del proyecto: repositorio CardGameLang.
- Gramática: `BisonGrammar.y`.
- Léxico: `FlexPatterns.l`.
- Análisis semántico: `SemanticValidator.c`.
- Generación de código: `Generator.c`.

7. Bibliografía

- Flex — *The Fast Lexical Analyzer*. <https://github.com/westes/flex>
- GNU Bison — *Bison Manual*. <https://www.gnu.org/software/bison/manual/>
- Aho, Lam, Sethi, Ullman. *Compilers: Principles, Techniques, and Tools* (2.^a ed.).
- CMake — *Documentation*. <https://cmake.org/documentation/>